COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	SECTION:	CPE21S2
DATE PERFORMED:	08-05-26	DATE SUBMITTED:	08-05-26
NAME:	Christian Lee C. Bandigan	INSTRUCTOR:	Engr. Jimlord Quejado

ACTIVITY NO. 5 Hands-on Activity 5.1 Queues

1. PROCEDURE OUTPUT

ILOA

HOA5 > ILOA > main.cpp > ...

```cpp
#include <iostream>
#include <queue>

void display(std::queue<char> Q);

int main(){

    //create an object:
    std::queue<char> myQ;

    //use the enqueue operation
    myQ.push('C');
    myQ.push('H');
    myQ.push('R');
    myQ.push('I');
    myQ.push('S');


    std::cout << "current front is: "<<myQ.front()<<std::endl;
    std::cout << "current back is: "<< myQ.back()<<std::endl;

    display(myQ);

    //use the dequeue operation
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);
    myQ.pop();
    display(myQ);

    return 0;
}
void display(std::queue<char> Q){
    std::queue<char> temp = Q;
    std::cout<<"\nThe contents of the queue:"<<std::endl;
    while(temp.size() != 0){
        std::cout << temp.front()<<" ";
        temp.pop();
    }
}
```

Figure 1.

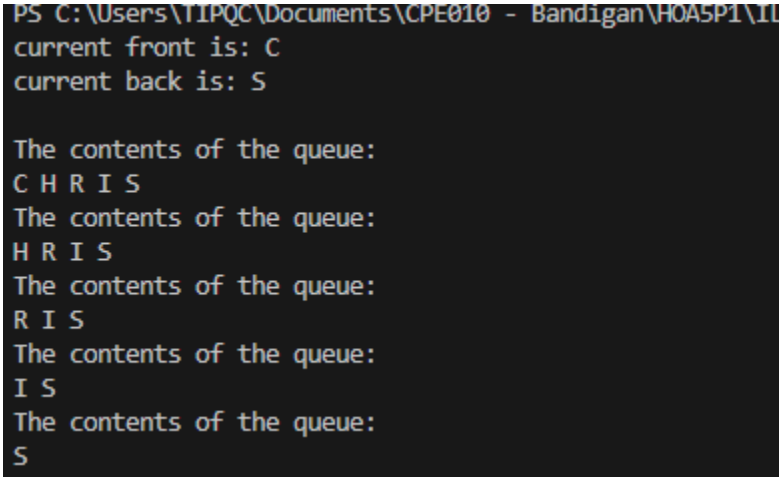


Figure 2.

Remarks

Using the STL in creating queues is really similar to when I made stacks and the difference is it tracks front and back instead of top. This code shows that you can use STL in combination with custom functions.

ILOB.1

HOA5P1 > ILOB > C queuell.h > isEmpty<T>(qNode<T>*, qNode<T>*)

```
#ifndef QUEUELL_H
#define QUEUELL_H

//create a class node
template <typename T>
class qNode{
    public:
        T data;
        qNode* next;
};
//new node creator
template <typename T>
qNode<T>* new_Node(T newData){
    //allocate  a space for the new node
    qNode<T>* newNode = new qNode<T>;

    //store the newData to the newNode
    newNode->data = newData;

    //point the newNode next to null
    newNode->next = nullptr;

    return newNode;
}
//enqueue
template <typename T>
void enqueue(T newData, qNode<T>** frontPtr, qNode<T>** backPtr){
    //create a new node
    qNode<T>* newNode = new_Node(newData);

    //check if the queue is empty
    if((*frontPtr)==nullptr && (*backPtr)==nullptr){
        (*frontPtr) = newNode;
        (*backPtr) = newNode;
        return;
    }

    //if not then insert at the back
    (*backPtr)->next = newNode; //LINKS the current back to the new node
    (*backPtr) = newNode;//ASSIGNs the backPtr to the new node
}
```

```
43  //dequeue
44  template<typename T>
45  void dequeue(qNode<T>** frontPtr, qNode<T>** backPtr){
46      //check if the queue is empty
47      if((*frontPtr) == nullptr && (*backPtr) == nullptr){
48          std::cout << "The queue is empty" << std::endl;
49          return;
50      }
51      //create a temporary vatiable to store the new node to be deleted
52      qNode<T>* deleteTemp = nullptr;
53
54      //assigns the current front to the deleteTemp
55      deleteTemp = (*frontPtr);
56
57      //check if the queue has only one item
58      if((*frontPtr)->next == nullptr && (*backPtr)->next==nullptr){
59          //means only one item is in the queue
60          (*frontPtr) = nullptr;
61          (*backPtr) = nullptr;
62          delete deleteTemp;
63          return;
64      }
65
66      (*frontPtr) = (*frontPtr)->next;
67      deleteTemp->next = nullptr;
68
69      //delete node
70      delete deleteTemp;
71  }
72
73  //isEmpty
74  template <typename T>
75  bool isEmpty(qNode<T>* frontPtr, qNode<T>* backPtr){
76      if (frontPtr== nullptr && backPtr==nullptr){
77          return true;
78      }else{
79          return false;
80      }
81  }
```

```
83  //diplayAll
84  template <typename T>
85  void display(qNode<T>* frontPtr, qNode<T>* backPtr){
86      if(frontPtr == nullptr && backPtr==nullptr){
87          std::cout << "The queue is empty" << std::endl;
88          return;
89      }
90
91      while(frontPtr != nullptr){
92          //PRINT data OF n
93          std::cout << frontPtr->data;
94          if(frontPtr->next != nullptr){
95              std::cout << " ";
96          }
97          //GO TO NEXT NODE n := next
98          frontPtr= frontPtr->next;
99      }
100     std::cout <<std::endl;
101 }
102
103 #endif
```

Figure 3.

HOA5P1 > ILOB > main.cpp > main()

```cpp
#include <iostream>
#include "queuell.h"

int main(){
    qNode<char>* front = nullptr;
    qNode<char>* back = nullptr;

    enqueue('C', &front, &back);
    std::cout <<"front: "<< front->data<<" back: "<<back->data<<std::endl;
    enqueue('H', &front, &back);
    std::cout <<"front: "<< front->data<<" back: "<<back->data<<std::endl;
    enqueue('R', &front, &back);
    std::cout <<"front: "<< front->data<<" back: "<<back->data<<std::endl;

    display(front,back);

    std::cout << "testing the dequeue operator: \n";
    dequeue(&front,&back);
    std::cout <<"front: "<< front->data<<" back: "<<back->data<<std::endl;
    dequeue(&front,&back);
    std::cout <<"front: "<< front->data<<" back: "<<back->data<<std::endl;
    dequeue(&front,&back);

    std::cout << "is the queue empty:" <<isEmpty(front,back)<<std::endl;

    return 0;
}
```

Figure 4.

```
PS C:\Users\TIPQC\Documents\CPE010 - Bandigan\HOA5P1\ILOB> cd "c:\Users\TIP
front: C back: C
front: C back: H
front: C back: R
C H R
testing the dequeue operator:
front: H back: R
front: R back: R
is the queue empty:1
PS C:\Users\TIPQC\Documents\CPE010 - Bandigan\HOA5P1\ILOB>
```

Figure 5.

Remarks

I have the same observation from before where implementing queues using linked lists is really similar with stacks but instead of using top you have to use front and back to enforce FIFO (first in first out).

ILOB.2

HOA5 > ILOB.2 > C queueArr.h > queueArr(const queueArr &)

```
#ifndef QUEUEARR_H
#define QUEUEARR_H

#include <iostream>

//Array-Based Circular Queue
template <typename T>
class queueArr{
    private:
        // pointer to dynamically allocate array
        T* q_array;
        //maximum number of elements queue can hold
        size_t q_capacity;
        //currrent number of elements in the queue
        size_t q_size;
        //Index of the front element
        int q_front;
        //Index of the back element
        int q_back;
    public:
        //constructor
        queueArr(size_t capacity = 10);
        //copy constructor
        queueArr(const queueArr& other);
        //copy assignment operator
        queueArr& operator=(const queueArr& other);
        //destructor
        ~queueArr();
        //queue operations:
        bool Empty();
        bool Full();
        size_t Size();
        void Clear();
        T front();
        T back();
        void Enqueue(T value);
        T Dequeue();
};
```

Figure 6. Class for Circular Array Queue

```cpp
//constructor
template <typename T>
queueArr<T>::queueArr(size_t capacity){
    q_capacity = capacity;
    q_array = new T[q_capacity];
    q_size = 0;
    q_front = 0;
    q_back = -1;
}

//copy constructor
template <typename T>
queueArr<T>::queueArr(const queueArr& other){
    q_capacity = other.capacity;
    q_array = new T[q_capacity];
    q_size = other.q_size;
    q_front = other.q_front;
    q_back = other.q_back;
    for (int i= 0; i < q_capacity;i++){
        q_array[i] = other.q_array[i];
    }
}
//copy assignment operator
template <typename T>
queueArr<T>& queueArr<T>::operator=(const queueArr& other){
    if (this != &other){
        delete[] q_array;

        q_capacity = other.q_capacity;
        q_array = new T[q_capacity];
        q_size = other.q_size;
        q_front = other.q_front;
        q_back = other.q_back;

        for (size_t i = 0; i <q_capacity; i++){
            q_array[i] = other.q_array[i];
        }
    }
    return *this;
}

//destructor
template <typename T>
queueArr<T>::~queueArr(){
    delete[] q_array;
}
```

Figure 7. Constructor and Big 3

Analysis

The constructor creates the object that initializes the needed attributes for a queue based array which are capacity, pointer to a new array, size, front, and back. The copy constructor initializes a newly created queue using the attributes of an existing queue. It copies the contents of the queue using a for loop. The copy assignment operator copies a queue to an existing queue by first checking if it is copying itself using an if statement. If not it will delete the contents of its current array then it uses the same code as the copy constructor. The destructor runs when the program ends to delete the object and the array to prevent memory leak

```cpp
//Empty
template <typename T>
bool queueArr<T>::Empty(){
    //check if the size is zero
    return q_size == 0;
}

//Full
template <typename T>
bool queueArr<T>::Full(){
    //check if the q_size reached its maximum capacity
    return q_size == q_capacity;
}

//find the currentt size of the array
template <typename T>
size_t queueArr<T>::Size(){
    return q_size;
}

//front
template <typename T>
T queueArr<T>::front(){
    //check if the queue is empty
    if(Empty()){
        std::cout << "Queue is empty." << std::endl;
        return T{};
    }
    //return the front
    return q_array[q_front];
}

//back
template <typename T>
T queueArr<T>::back(){
    if(Empty()){
        std::cout <<"Queue is empty."<<std::endl;
        return T{};
    }
    return q_array[q_back];
}
```

Figure 8. Check Functions

Analysis

These are the check functions for the queue. They just check if the queue is empty or full and return the size, front element, and back element of the queue.

```
129  //clear
130  template <typename T>
131  void queueArr<T>::Clear(){
132      q_size =  0;
133      q_front = 0;
134      q_back = -1;
135  }
136
137  template <typename T>
138  void queueArr<T>::Enqueue(T value){
139      //check if the queue is full
140      if(Full()){
141          std::cout <<"Queue is full."<<std::endl;
142          return;
143      }
144      //move q_back circularly
145      q_back = (q_back + 1) % q_capacity;
146      // store the value to the back
147      q_array[q_back] = value;
148      //increment the q_size
149      q_size++;
150  }
151
152  template <typename T>
153  T queueArr<T>::Dequeue(){
154      //check if empty
155      if(Empty()){
156          std::cout <<"Queue is empty."<<std::endl;
157          return T{};
158      }
159      //create a temporary variable to store the current front
160      T temp = q_array[q_front];
161      //move q_front circularly
162      q_front = (q_front + 1) % q_capacity;
163      //decrement the q_size
164      q_size--;
165      //reset the indexes if the queue becomes empty
166      if(Empty()) Clear();
167      //return the temporary variable
168      return temp;
169  }
170
171
172  #endif
```

Figure 9. Clear, Enqueue, Dequeue Functions

Analysis

The clear function clears the array of all the elements by resetting the indexes. The enqueue function just adds onto the back index and size to to add an element in the queue. The dequeue function dequeues the front element of the queue by adding onto the front index and decrementing the size. Since this implementation uses a circular array, to move the index forward you have to use modulo so when it reaches the end of the array it will go to the front since the remainder will be zero.

HOA5 > ILOB.2 > main.cpp > ...

```cpp
#include <iostream>
#include "queueArr.h"

int main(){

    //creting an object
    queueArr<int> q(5);

    //enqueue 10,20,30,40,50
    q.Enqueue(1);

    std::cout << "Front: " <<q.front()<<std::endl;
    std::cout << "Back: " <<q.back()<<std::endl;
    std::cout << "Size: " <<q.Size()<<std::endl;

    q.Enqueue(2);
    std::cout << "Front: " <<q.front()<<std::endl;
    std::cout << "Back: " <<q.back()<<std::endl;
    std::cout << "Size: " <<q.Size()<<std::endl;
    q.Enqueue(3);
    q.Enqueue(4);
    std::cout << "Front: " <<q.front()<<std::endl;
    std::cout << "Back: " <<q.back()<<std::endl;
    std::cout << "Size: " <<q.Size()<<std::endl;
    q.Enqueue(5);
    std::cout << "Front: " <<q.front()<<std::endl;
    std::cout << "Back: " <<q.back()<<std::endl;
    std::cout << "Size: " <<q.Size()<<std::endl;
    q.Enqueue(6);

    std::cout <<"removed: "<< q.Dequeue()<< std::endl;
    std::cout <<"removed: "<< q.Dequeue()<< std::endl;

    q.Clear();
    std::cout << "is the queue empty? " << (q.Empty() ? "yes":"no")<<std::endl;

    return 0;
}
```

Figure 10. Main Code

```
PS C:\Users\User\OneDrive\Documents\CPE010_DSA> cd "c:\Users
main }
Front: 1
Back: 1
Size: 1
Front: 1
Back: 2
Size: 2
Front: 1
Back: 4
Size: 4
Front: 1
Back: 5
Size: 5
Queue is full.
removed: 1
removed: 2
is the queue empty? yes
PS C:\Users\User\OneDrive\Documents\CPE010_DSA\HOA5\ILOB.2>
```

Figure 11. Output

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

HOA5 > ILOC > C C_queuell.h > ...

```cpp
#ifndef C_QUEUELL_H
#define C_QUEUELL_H
#include <string>

class Job{
    public:
        int Id;
        std::string name;
        int pages;
        Job(int newId = 0,std::string newName = "", int newPages = 0){
            Id = newId;
            name = newName;
            pages = newPages;
        }
};

class qNode{
    public:
        Job data;
        qNode* next;
};

qNode* new_Node(Job newData){
    //allocate  a space for the new node
    qNode* newNode = new qNode;

    //store the newData to the newNode
    newNode->data = newData;

    //point the newNode next to null
    newNode->next = nullptr;

    return newNode;
}
```

Figure 12. Job and Node Class and New Node Function

Analysis

The job class creates an object that has attributes for a printing job. I modified the class node and the new node function so it would take an object from the job class.

```
36  class Printer{
37      private:
38          qNode* front;
39          qNode* back;
40      public:
41          Printer(){
42              front = nullptr;
43              back = nullptr;
44          }
45
46          bool isEmpty(){
47              if (front== nullptr && back==nullptr){
48                  return true;
49              }else{
50                  return false;
51              }
52          }
53          //enqueue
54          void addJob(Job job){
55              //create a new node
56              qNode* newNode = new_Node(job);
57
58              //check if the queue is empty
59              if(isEmpty()){
60                  front = newNode;
61                  back = newNode;
62                  return;
63              }
64              //if not then insert at the back
65              back->next = newNode; //LINKS the current back to the new node
66              back = newNode;//ASSIGNs the backPtr to the new node
67          }
```

Figure 13. Printer Class

Analysis

I put the creation and initialization of the front and back pointers and the functions in this class so in the main driver you just need to make an object for the printer. The addJob function is just the modified enqueue function. It expects a job object and since the front and back pointer is created in the class you don't need a dereference operator.

```
void processJob(){
    // check empty
    if(isEmpty()){
        std::cout << "No jobs to print." << std::endl;
        return;
    }
    //create a temporary vatiable to store the new node to be deleted
    qNode* temp = nullptr;
    //assigns the current front to the deleteTemp
    temp = front;
    //display job being printed
    std::cout <<std::endl;
    std::cout << "Printing:" << std::endl;
    std::cout << "Job ID: " << front->data.Id<< std::endl;
    std::cout << "User: " << front->data.name << std::endl;
    std::cout << "Pages: " << front->data.pages << std::endl;

    //check if the queue has only one item
   if(front->next == nullptr && back->next==nullptr){
        //means only one item is in the queue
        front = nullptr;
        back = nullptr;
        delete temp;
        return;
    }
    front = front->next;
    temp->next = nullptr;

    //delete node
    delete temp;
    std::cout << "Job Printed." << std::endl;
}

void display(){
    if(isEmpty()){
        std::cout << "The queue is empty" << std::endl;
        return;
    }
    qNode* temp = front;
    std::cout << "================ Current Queue ================"<< std::endl;
    while(temp != nullptr){
        //PRINT data OF n
        std::cout << "Job ID: " << temp->data.Id << "      User: " << temp->data.name << "      Pages: " << temp->data.pages << std::endl;
        //GO TO NEXT NODE n := next
        temp= temp->next;
    }
}
};
#endif
```

Figure 14. Process and Display Functions

Analysis

The process job function is just the modified dequeue function. It prints the attributes of the printing job being processed. In the display function I just added a temporary pointer to the front to traverse the list since the front is in the class.

HOA5 > ILOC > mainC.cpp > ...

```cpp
#include <iostream>
#include "C_queuell.h"

int main(){
    Printer printer;

    printer.addJob(Job(1,"Christian",3));
    printer.addJob(Job(2,"Topher",1));
    printer.addJob(Job(3,"Acquila",3));
    printer.addJob(Job(4,"Gianne",5));

    printer.display();
    printer.processJob();
    printer.processJob();

    printer.addJob(Job(5,"Justin",4));
    printer.addJob(Job(6,"Carlito",7));
    printer.addJob(Job(7,"Brylle",3));
    printer.addJob(Job(8,"Louis",2));
    printer.display();

    printer.processJob();
    printer.processJob();
    printer.display();

    return 0;
}
```

Figure 15. Main

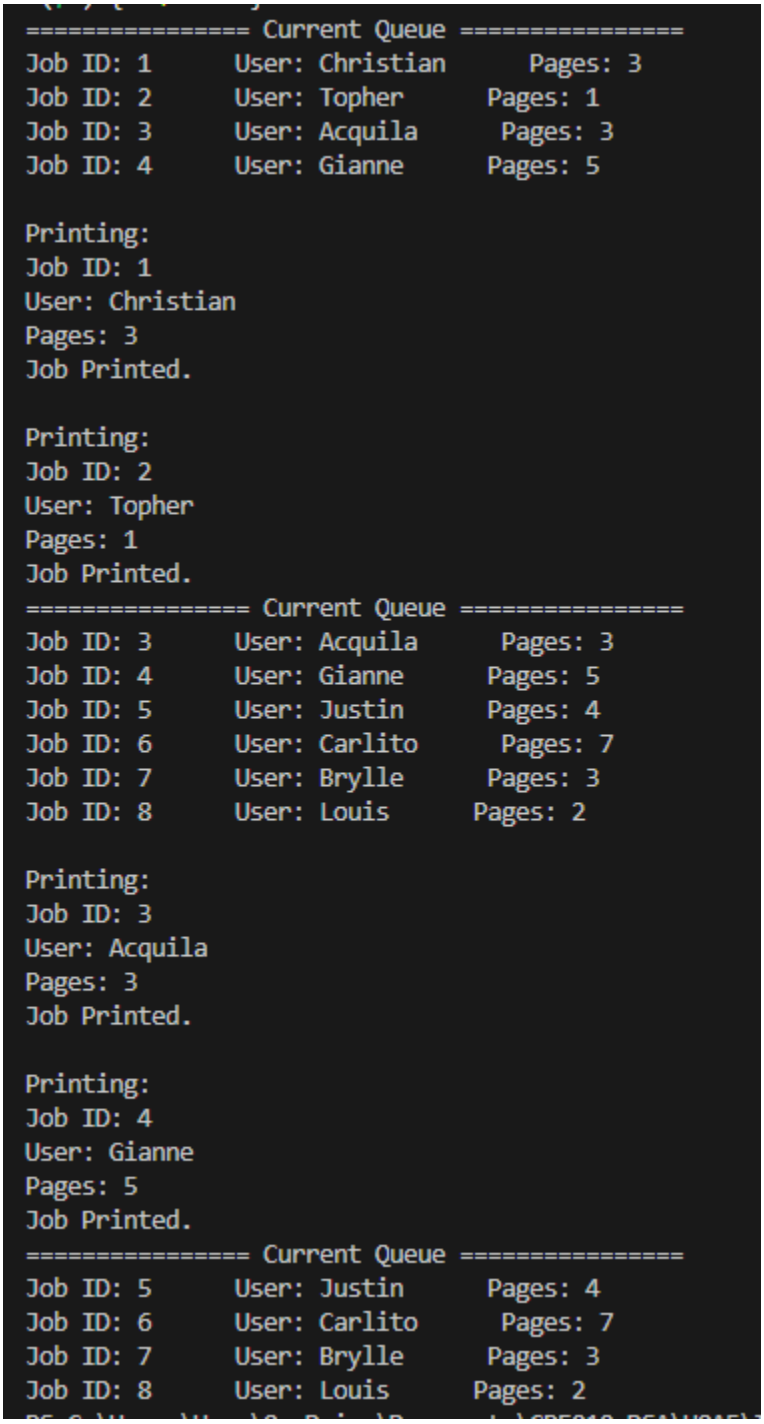


Figure 16. Output

I used a linked list implementation because I think it is easier to implement since you only need to account for the front and back pointers and you can add and process jobs without moving elements. Another reason is linked lists have dynamic space. You don't need to worry about the queue being full or if it is wasting memory.

Conclusion

In this activity I learned about queues and how to implement them in three ways using the C++ STL, linked lists, and arrays. A queue is a data structure that operates on FIFO (First in, First Out), so in code you have to account for the element at the front and back. I was able to create operations that handle queues like enqueue, dequeue, empty, display all, etc. Moreover, I was able to solve a problem about shared printers in an office using a linked list based queue implementation.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: _____

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: _____

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: _____

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.